

Multimodal AI Pipeline for Breast Tumor Classification

This notebook explores how deep learning and large language models can be combined to support medical imaging analysis.

The system uses:

- **BreastMNIST dataset** (MedMNIST)
- **CNN model in PyTorch** for tumor classification
- **Qwen LLM** to generate interpretable explanations of results

Goal: explore how multimodal AI systems can improve interpretability in medical AI.

Author: Maryam Shahbaz Ali

Notebook Structure

1. Environment Setup
2. Dataset Loading (BreastMNIST)
3. Data Preprocessing
4. CNN Model Architecture
5. Model Training
6. Model Evaluation
7. LLM-based Explanation Generation

Environment Setup

Run the following cell in Google Colab to install the required packages. Restart the runtime once if prompted.

```
# Install dependencies
!pip -q install medmnist torch torchvision transformers accelerate tqdm matplotlib scikit-learn pillow
```

```
# Core libraries
import os
import sys
import platform
import requests
import numpy as np
import matplotlib.pyplot as plt

# Deep learning
import torch
import torch.nn as nn
import torch.optim as optim
import torch.utils.data as data
import torchvision.transforms as transforms
from torch.utils.data import DataLoader
```

```
# Dataset and utilities
from tqdm import tqdm
import medmnist
from medmnist import BreastMNIST, INFO

# Evaluation
from sklearn.metrics import roc_auc_score, confusion_matrix, accuracy_score, classification_report

# LLM tools
from transformers import AutoTokenizer, AutoModelForCausalLM
```

Environment Check

This section verifies the runtime configuration and hardware availability.

```
print("Python:", sys.version)
print("Platform:", platform.platform())
print("PyTorch:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
```

```
Python: 3.12.12 (main, Oct 10 2025, 08:52:57) [GCC 11.4.0]
Platform: Linux-6.6.113+-x86_64-with-glibc2.35
PyTorch: 2.10.0+cpu
CUDA available: False
Using device: cpu
```

Dataset: BreastMNIST

BreastMNIST is a binary classification dataset from MedMNIST containing breast ultrasound images labeled as benign or malignant.

```
!pip install medmnist
```

```
Requirement already satisfied: medmnist in /usr/local/lib/python3.12/dist-packages (3.0.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from medmnist) (2.0.2)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from medmnist) (2.2.2)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (from medmnist) (1.8.0)
Requirement already satisfied: scikit-image in /usr/local/lib/python3.12/dist-packages (from medmnist) (0.25.2)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from medmnist) (4.67.3)
Requirement already satisfied: Pillow in /usr/local/lib/python3.12/dist-packages (from medmnist) (12.1.1)
Requirement already satisfied: fire in /usr/local/lib/python3.12/dist-packages (from medmnist) (0.7.1)
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (from medmnist) (2.10.0+cpu)
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-packages (from medmnist) (0.25.0+cpu)
Requirement already satisfied: termcolor in /usr/local/lib/python3.12/dist-packages (from fire->medmnist) (3.3.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->medmnist) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->medmnist) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->medmnist) (2025.3)
Requirement already satisfied: scipy>=1.11.4 in /usr/local/lib/python3.12/dist-packages (from scikit-image->medmnist) (1.16.3)
Requirement already satisfied: networkx>=3.0 in /usr/local/lib/python3.12/dist-packages (from scikit-image->medmnist) (3.6.1)
Requirement already satisfied: imageio!=2.35.0,>=2.33 in /usr/local/lib/python3.12/dist-packages (from scikit-image->medmnist) (2.37.2)
```

```
Requirement already satisfied: tifffile>=2022.8.12 in /usr/local/lib/python3.12/dist-packages (from scikit-image->medmnist) (2026.2.24)
Requirement already satisfied: packaging>=21 in /usr/local/lib/python3.12/dist-packages (from scikit-image->medmnist) (26.0)
Requirement already satisfied: lazy-loader>=0.4 in /usr/local/lib/python3.12/dist-packages (from scikit-image->medmnist) (0.4)
Requirement already satisfied: joblib>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn->medmnist) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn->medmnist) (3.6.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch->medmnist) (3.24.3)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch->medmnist) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch->medmnist) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch->medmnist) (1.14.0)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch->medmnist) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch->medmnist) (2025.3.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas->medmnist) (1.17.0)
Requirement already satisfied: mpmath<1.4, >=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch->medmnist) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->torch->medmnist) (3.0.3)
```

```
import os, sys, platform, torch
print("Python:", sys.version)
print("Platform:", platform.platform())
print("Torch:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())
if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
```

```
Python: 3.12.12 (main, Oct 10 2025, 08:52:57) [GCC 11.4.0]
Platform: Linux-6.6.113+-x86_64-with-glibc2.35
Torch: 2.10.0+cpu
CUDA available: False
```

Dataset: BreastMNIST

BreastMNIST is a binary classification dataset from MedMNIST containing breast ultrasound images labeled as benign or malignant.

```
from medmnist import BreastMNIST
train_dataset = BreastMNIST(download=True, split="train")
```

```
train_dataset
```

```
Dataset BreastMNIST of size 28 (breastmnist)
  Number of datapoints: 546
  Root location: /root/.medmnist
  Split: train
  Task: binary-class
  Number of channels: 1
  Meaning of labels: {'0': 'malignant', '1': 'normal, benign'}
  Number of samples: {'train': 546, 'val': 78, 'test': 156}
  Description: The BreastMNIST is based on a dataset of 780 breast ultrasound images. It is categorized into 3 classes: normal, benign, and malignant. As we use low-resolution images, we simplify the task into binary classification by combining normal and benign as positive and classifying them against malignant as negative. We split the source dataset with a ratio of 7:1:2 into training, validation and test set. The source images of 1x500x500 are resized into 1x28x28.
  License: CC BY 4.0
```

```
len(train_dataset)
```

```
546
```

```
!pip -q install -U medmnist tqdm scikit-learn matplotlib pillow requests
```

```
data_flag = 'breastmnist'
download = True

NUM_EPOCHS = 3
BATCH_SIZE = 128
lr = 0.001

info = INFO[data_flag]
task = info['task']
n_channels = info['n_channels']
n_classes = len(info['label'])

DataClass = getattr(medmnist, info['python_class'])

print("Dataset:", data_flag)
print("Task:", task)
print("N channels:", n_channels)
print("Labels:", info['label'])
print(f"MedMNIST v{medmnist.__version__} @ {medmnist.HOMEPAGE}")
```

```
Dataset: breastmnist
Task: binary-class
N channels: 1
Labels: {'0': 'malignant', '1': 'normal, benign'}
MedMNIST v3.0.2 @ https://github.com/MedMNIST/MedMNIST/
```

```
transform = transforms.Compose([
    transforms.ToTensor(),
])

train_dataset = DataClass(split='train', transform=transform, download=download)
val_dataset = DataClass(split='val', transform=transform, download=download)
test_dataset = DataClass(split='test', transform=transform, download=download)

train_loader = data.DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
val_loader = data.DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False)
test_loader = data.DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)

print("Train samples:", len(train_dataset))
print("Validation samples:", len(val_dataset))
print("Test samples:", len(test_dataset))
```

```
Train samples: 546
Validation samples: 78
Test samples: 156
```

```
# Sample montage
train_dataset.montage(length=20)
plt.show()
```

CNN Model Architecture

A small convolutional neural network is used as a baseline classifier for benign vs malignant prediction.

```
# Defining a small CNN (binary classification)

class SmallCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 32, 3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2), # 14x14
            nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2), # 7x7
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64*7*7, 128), nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(128, 1) # logits
        )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x.squeeze(1)

model_cnn = SmallCNN().to(device)
print(model_cnn)
```

```
SmallCNN(
  (features): Sequential(
    (0): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): ReLU()
    (2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (3): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (4): ReLU()
    (5): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (classifier): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=3136, out_features=128, bias=True)
    (2): ReLU()
    (3): Dropout(p=0.2, inplace=False)
    (4): Linear(in_features=128, out_features=1, bias=True)
  )
)
```

Model Training

Train the CNN using binary cross-entropy loss and the Adam optimizer. The best checkpoint is selected using validation ROC-AUC.

```
# Train loop + evaluation helpers
from sklearn.metrics import roc_auc_score, confusion_matrix, accuracy_score

def eval_loader(model, loader):
    model.eval()
    all_logits = []
    all_y = []
```

```

with torch.no_grad():
    for x, y in loader:
        x = x.to(device)
        y = y.to(device).float().view(-1)
        logits = model(x)
        all_logits.append(logits.detach().cpu())
        all_y.append(y.detach().cpu())
logits = torch.cat(all_logits).numpy()
y_true = torch.cat(all_y).numpy().astype(int)

probs = 1 / (1 + np.exp(-logits))
y_pred = (probs >= 0.5).astype(int)

acc = accuracy_score(y_true, y_pred)
auc = roc_auc_score(y_true, probs) if len(np.unique(y_true)) == 2 else float("nan")
cm = confusion_matrix(y_true, y_pred)
return acc, auc, cm, probs, y_true

criterion = nn.BCEWithLogitsLoss()
optimizer = optim.Adam(model_cnn.parameters(), lr=1e-3)

EPOCHS = 5
best_val_auc = -1.0

for epoch in range(1, EPOCHS + 1):
    model_cnn.train()
    pbar = tqdm(train_loader, desc=f"Epoch {epoch}/{EPOCHS}", leave=False)
    for x, y in pbar:
        x = x.to(device)
        y = y.to(device).float().view(-1)
        optimizer.zero_grad()
        logits = model_cnn(x)
        loss = criterion(logits, y)
        loss.backward()
        optimizer.step()
        pbar.set_postfix(loss=float(loss.item()))

    val_acc, val_auc, val_cm, _, _ = eval_loader(model_cnn, val_loader)
    print(f"Epoch {epoch}: val_acc={val_acc:.4f} val_auc={val_auc:.4f}")
    print("Val confusion matrix:\n", val_cm)

    if val_auc > best_val_auc:
        best_val_auc = val_auc
        torch.save(model_cnn.state_dict(), "best_cnn.pt")
        print("Saved best model to best_cnn.pt")

```

```

Epoch 1: val_acc=0.7308 val_auc=0.5539
Val confusion matrix:
[[ 0 21]
 [ 0 57]]
Saved best model to best_cnn.pt
Epoch 2: val_acc=0.7308 val_auc=0.5957
Val confusion matrix:
[[ 0 21]
 [ 0 57]]
Saved best model to best_cnn.pt
Epoch 3: val_acc=0.7308 val_auc=0.6007
Val confusion matrix:

```

```

[[ 0 21]
 [ 0 57]]
Saved best model to best_cnn.pt
Epoch 4: val_acc=0.7308 val_auc=0.6491
Val confusion matrix:
[[ 0 21]
 [ 0 57]]
Saved best model to best_cnn.pt
Epoch 5: val_acc=0.7308 val_auc=0.6834
Val confusion matrix:
[[ 0 21]
 [ 0 57]]
Saved best model to best_cnn.pt

```

Model Evaluation

Evaluate the best model checkpoint on the test set using accuracy, ROC-AUC, and a confusion matrix.

```

# Final test evaluation (best checkpoint)
model_cnn.load_state_dict(torch.load("best_cnn.pt", map_location=device))
test_acc, test_auc, test_cm, test_probs, test_y = eval_loader(model_cnn, test_loader)

print(f"TEST: acc={test_acc:.4f} auc={test_auc:.4f}")
print("Test confusion matrix:\n", test_cm)

print("\nClassification report:")
print(classification_report(test_y.astype(int), (test_probs >= 0.5).astype(int), digits=4))

```

```

TEST: acc=0.7308 auc=0.7055
Test confusion matrix:
[[ 0 42]
 [ 0 114]]

```

```

Classification report:

```

	precision	recall	f1-score	support
0	0.0000	0.0000	0.0000	42
1	0.7308	1.0000	0.8444	114
accuracy			0.7308	156
macro avg	0.3654	0.5000	0.4222	156
weighted avg	0.5340	0.7308	0.6171	156

```

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1833: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predic
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1833: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predic
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1833: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predic
_warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])

```

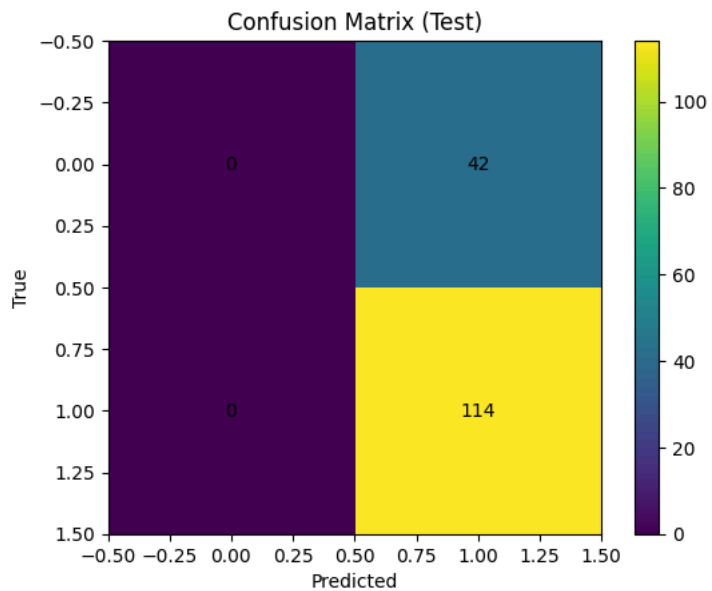
```

# Confusion matrix visualization
import matplotlib.pyplot as plt
import numpy as np

plt.figure()
plt.imshow(test_cm)
plt.title("Confusion Matrix (Test)")
plt.xlabel("Predicted")

```

```
plt.ylabel("True")
plt.colorbar()
for (i, j), v in np.ndenumerate(test_cm):
    plt.text(j, i, str(v), ha="center", va="center")
plt.show()
```



Transformers for Qwen

```
!pip -q install -U transformers accelerate
```

```
# Load Qwen via Transformers
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

QWEN_HF_MODEL = "Qwen/Qwen2.5-1.5B-Instruct"

tokenizer = AutoTokenizer.from_pretrained(QWEN_HF_MODEL, trust_remote_code=True)
model = AutoModelForCausalLM.from_pretrained(
    QWEN_HF_MODEL,
    torch_dtype=torch.float16 if torch.cuda.is_available() else torch.float32,
    device_map="auto",
    trust_remote_code=True
)

def qwen_hf(prompt, max_new_tokens=200):
    inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
    out = model.generate(**inputs, max_new_tokens=max_new_tokens, do_sample=True, temperature=0.7)
    return tokenizer.decode(out[0], skip_special_tokens=True)
```

```
print(qwen_hf("Write a short, clinician-friendly definition of sensitivity and specificity."))
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
```

The secret `HF\_TOKEN` does not exist in your Colab secrets.

To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set it as secret in your Google Colab and restart your notebook. You will be able to reuse this secret in all of your notebooks.

Please note that authentication is recommended but still optional to access public models or datasets.

```
warnings.warn(
`torch_dtype` is deprecated! Use `dtype` instead!
```

```
model.safetensors: 100% 3.09G/3.09G [00:42<00:00, 111MB/s]
```

```
Loading weights: 100% 338/338 [00:12<00:00, 20.22it/s]
```

```
generation_config.json: 100% 242/242 [00:00<00:00, 18.2kB/s]
```

Write a short, clinician-friendly definition of sensitivity and specificity. Sensitivity refers to the proportion of actual cases that are correctly identified by a test or score.

```
print(f"MedMNIST v{medmnist.__version__} @ {medmnist.HOMEPAGE}")
```

```
MedMNIST v3.0.2 @ https://github.com/MedMNIST/MedMNIST/
```

```
# visualization
```

```
train_dataset.montage(length=20)
```



```
# montage
```

```
train_dataset.montage(length=20)
```



```
train_dataset.labels
```

```
[1],  
[1],  
[1],  
[1],  
[0],  
[1],  
[0],  
[1],  
[1],  
[0],  
[1],  
[1],  
[1],  
[1],  
[1],  
[1],  
[0],  
[1],  
[1],  
[1],  
[1],  
[1],  
[1],  
[1],  
[0],  
[1],  
[0],  
[0],  
[1],  
[1],  
[1],  
[1],  
[1],  
[1],  
[1],  
[1],  
[0]], dtype=uint8)
```

Using Qwen to write a short summary

```
import torch  
from transformers import AutoTokenizer, AutoModelForCausalLM  
  
model_name = "Qwen/Qwen2.5-3B-Instruct"  
  
tokenizer = AutoTokenizer.from_pretrained(model_name)  
model = AutoModelForCausalLM.from_pretrained(  
    model_name,  
    torch_dtype="auto",  
    device_map="auto"  
)  
  
print("Qwen loaded successfully with Hugging Face Transformers")
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
config.json: 100% 661/661 [00:00<00:00, 78.8kB/s]

tokenizer_config.json: 7.30k/? [00:00<00:00, 687kB/s]
vocab.json: 2.78M/? [00:00<00:00, 66.8MB/s]
merges.txt: 1.67M/? [00:00<00:00, 56.2MB/s]
tokenizer.json: 7.03M/? [00:00<00:00, 126MB/s]
model.safetensors.index.json: 35.6k/? [00:00<00:00, 3.36MB/s]
Download complete: 100% 6.17G/6.17G [08:14<00:00, 10.0MB/s]

Fetching 2 files: 100% 2/2 [08:13<00:00, 493.89s/it]
Loading weights: 100% 434/434 [00:01<00:00, 355.60it/s]

generation_config.json: 100% 242/242 [00:00<00:00, 22.7kB/s]
WARNING:accelerate.big_modeling:Some parameters are on the meta device because they were offloaded to the cpu and disk.
Qwen loaded successfully with Hugging Face Transformers
```

```
!pip -q install transformers accelerate sentencepiece
```

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

model_name = "Qwen/Qwen2.5-1.5B-Instruct"

tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    device_map="auto",
    torch_dtype="auto"
)
```

```
Loading weights: 100% 338/338 [00:00<00:00, 364.77it/s]
```

```
def qwen_hf(prompt, max_new_tokens=220):
    messages = [
        {
            "role": "system",
            "content": "You are assisting a medical imaging research trainee. Write clearly, cautiously, and in clinician-friendly language."
        },
        {
            "role": "user",
            "content": prompt
        }
    ]

    text = tokenizer.apply_chat_template(
        messages,
        tokenize=False,
        add_generation_prompt=True
    )
```

```

inputs = tokenizer(text, return_tensors="pt").to(model.device)

outputs = model.generate(
    **inputs,
    max_new_tokens=max_new_tokens,
    do_sample=True,
    temperature=0.7
)

response = tokenizer.decode(
    outputs[0][inputs["input_ids"].shape[1]:],
    skip_special_tokens=True
)
return response

```

```

# BreastMNIST label mapping:
# 0 = malignant
# 1 = normal / benign

tn_like = int(test_cm[0, 0]) # malignant predicted malignant
fn = int(test_cm[0, 1])      # malignant predicted benign
fp = int(test_cm[1, 0])      # benign predicted malignant
tp_like = int(test_cm[1, 1]) # benign predicted benign

results_text = (
    "BreastMNIST CNN baseline results:\n"
    f"- Test Accuracy: {test_acc:.4f}\n"
    f"- Test ROC-AUC: {test_auc:.4f}\n"
    f"- Confusion Matrix: {test_cm.tolist()}\n"
    f"- Label mapping: 0 = malignant, 1 = normal/benign\n"
    f"- Malignant correctly predicted as malignant: {tn_like}\n"
    f"- Malignant misclassified as benign (false negatives): {fn}\n"
    f"- Benign misclassified as malignant (false positives): {fp}\n"
    f"- Benign correctly predicted as benign: {tp_like}\n"
    f"- Total test samples: {len(test_y)}\n"
)

prompt = f"""
You are assisting a medical imaging research trainee.

Write a concise 120-160 word summary of the CNN results for a research supervisor.

Requirements:
- Use clinician-friendly language
- Mention test accuracy and ROC-AUC
- Explain the confusion matrix in plain terms
- Explicitly state that malignant cases predicted as benign are false negatives
- Highlight why false negatives are important in cancer detection
- Use the exact counts provided below
- Do not invent numbers
- Avoid hype
- Do not claim clinical deployment or clinical readiness

Results:
{results_text}
"""

```

```
summary = qwen_hf(prompt, max_new_tokens=220)
print(summary)
```

The BreastMNIST CNN model achieved an overall test accuracy of 73% and a ROC-AUC score of 70%, indicating moderate performance on the dataset. The confusion matrix shows that

- 0 were correctly identified as malignant (true negatives),
- 42 were incorrectly classified as benign (false negatives),
- 0 were incorrectly classified as malignant (false positives),
- 114 were correctly identified as benign (true positives).

This means that 42 out of the 156 malignant cases were misclassified as benign, which is a significant number given the importance of accurate cancer classification. This high